

Unit-2: Constraint-Based Problem Solving

Q1. Doctor Shift Assignment using Backtracking Search

1. Problem Understanding

Assign three doctors (D1, D2, D3) to three shifts:

- Morning (M)
- Afternoon (A)
- Night (N)

Each doctor must be assigned exactly one shift while satisfying all constraints.

2. Constraint Analysis

Constraint	Meaning
D1 cannot work Night	$D1 \neq \text{Night}$
D2 must work before D3	$\text{Shift}(D2) < \text{Shift}(D3)$
One doctor per shift	No duplicate shifts
D3 cannot work Morning	$D3 \neq \text{Morning}$
Every doctor gets one shift	Complete assignment
Shift Order:	
$\text{Morning} < \text{Afternoon} < \text{Night}$	

3. Solution Development (Backtracking)

Step 1: Assign D1

Possible shifts:

- Morning ✓
- Afternoon ✓
- Night ✗ (Constraint i)

Choose

$D1 \rightarrow \text{Morning}$

Current Assignment

Doctor Shift

D1 Morning

Constraint Check

- D1 not Night ✓

Step 2: Assign D2

Remaining shifts:

- Afternoon
- Night

Try

D2 → Afternoon

Current Assignment

Doctor Shift

D1 Morning

D2 Afternoon

Constraint Check

- Unique shifts ✓

Step 3: Assign D3

Remaining shift:

- Night

Check constraints

- D3 ≠ Morning ✓
- D2 before D3

Afternoon < Night ✓

Valid

Assignment completed.

(Alternative Branch)

Suppose

D1 → Afternoon

Remaining:
Morning, Night

Try

D2 → Morning

Remaining:
Night

D3 → Night

Checks

- D3 not Morning ✓
- Morning < Night ✓

Also valid.

4. Application of AI Concepts

State Space

Different combinations of doctor-shift assignments.

Initial State

No doctor assigned.

Goal State

All doctors assigned while satisfying constraints.

Backtracking

- Assign one doctor.
- Check constraints.

5. Justification

Backtracking is suitable because:

- Eliminates invalid assignments early.
- Avoids checking every permutation.
- Efficient for Constraint Satisfaction Problems (CSP).

Q2. Robot Navigation using BFS

1. Problem Understanding

Robot moves in a **5×5 grid**.

Movement:

- Up
- Down
- Left
- Right

Cost per move = 1

Cannot cross obstacles.

Need shortest path from Start (S) to Goal (G).

Grid

S	.	.	X	.
.	X	.	X	.
.	X	.	.	.
.	.	X	X	.
.	.	.	G	.

Legend

- S = Start
- G = Goal
- X = Obstacle

Coordinates

S = (0,0)

G = (4,3)

3. Solution Development (BFS)

Queue

Initially

Queue=[(0,0)]

Visited={ (0,0) }

Step 1

Expand (0,0)

Possible(1,0) (0,1)

Queue [(1,0),(0,1)]

Step 2

Expand (1,0)

New node (2,0)

Queue

[(0,1),(2,0)]

Step 3

Expand (0,1)

New node (0,2)

Queue

[(2,0),(0,2)]

Step 4

Expand (2,0)

New node (3,0)

Queue

[(0,2),(3,0)]

Step 5

Expand (0,2)

New node (1,2)

Queue

[(3,0),(1,2)]

Continue similarly.

Eventually Goal reached.

Optimal Path

(0,0)
↓
(1,0)
↓
(2,0)
↓

(3,0)
→
(3,1)
↓
(4,1)
→
(4,2)
→
(4,3)

Total Cost

Moves

8

Cost

$$8 \times 1 = 8$$

2. Constraint Analysis

Constraint	Effect
------------	--------

Up Down Left Right	Only four directions
--------------------	----------------------

Cost = 1	Equal edge cost
----------	-----------------

Obstacles	Cannot enter blocked cells
-----------	----------------------------

Manhattan Distance	Used to estimate distance to goal
--------------------	-----------------------------------

Manhattan Distance

$$h = |x_1 - x_2| + |y_1 - y_2|$$

Example

For Start

$$h = |0-4| + |0-3| = 7$$

4. Application of AI Concepts

- State = Robot location
- Initial State = Start
- Goal State = Goal
- BFS explores nodes level by level.
- Guarantees shortest path when move costs are equal.

5. Justification

BFS is appropriate because:

- Every move has equal cost.
- Finds the shortest path.
- Simple and complete.
- Avoids revisiting explored nodes.

Q3. Autonomous Rescue Robot

(a) Problem Understanding

A rescue robot must locate survivors inside a disaster-affected building.

The building is represented as a grid.

The robot:

- Starts at **S**
- Must reach **G**
- Faces blocked paths
- Has limited sensing
- Must avoid risky areas
- Must minimize total travel cost

Constraint Analysis

Constraint	Effect on Robot Decision
Move in four directions	Limits possible actions
Obstacles	Robot cannot enter blocked rooms
Limited sensing	Robot only knows adjacent cells and must explore gradually
Move cost = 1	Encourages shorter paths
Risk zone cost = +2	Robot prefers safer paths, even if slightly longer
Online updates	Robot replans when new obstacles are discovered

(b) Solution Development

Search Strategy

Uniform Cost Search (UCS) with Online Search

Step 1

Start from S.

Step 2

Sense neighboring rooms.

Unknown cells remain unexplored.

Step 3

Calculate total path cost.

Normal room = 1

Risk room = 3

Step 4

Expand the node with the smallest cumulative cost.

Step 5

If a blocked path is discovered

- Update the map
- Remove blocked path
- Search again

Step 6

Repeat until survivor (G) is reached.

Example Cost Calculation

Safe path

$$1+1+1+1=4$$

Risk path

$$1+3+1=5$$

Robot chooses

Safe path (Cost = 4)

(c) Application of AI Concepts

Intelligent Agent

The rescue robot:

- Senses environment
- Makes decisions
- Plans actions
- Learns new obstacles
- Acts autonomously

Rationality

A rational agent always selects the action with the **lowest expected total cost** while ensuring safety.

Environment Type

Property	Type
Partial observability	Robot sees only nearby cells
Dynamic	Environment may change due to debris
Sequential	Each action affects future choices
Discrete	Building represented as grid cells
Unknown	Robot discovers obstacles during movement

(d) Justification

Uniform Cost Search combined with an Online Search Agent is the most suitable approach because:

- Finds the **lowest-cost path**, considering both movement and risk costs.
- Handles **different path costs** effectively.
- Supports **dynamic replanning** when new obstacles are detected.
- Works well in **partially observable environments** where the robot gradually discovers the map.